

PER2-HALO: QC peptide-level quantification and filter

Tom Smith

2026-01-30

Here, we will read in the peptide-level quantification data and perform routine filtering.

Setup

Load required packages

```
library(biomasslmb)
library(ggplot2)
library(tidyr)
library(dplyr)
library(here)
library(QFeatures)
library(xml2)
```

Read in the protein details

```
uniprot2details <- readRDS(here('external/uniprot2details_unique.rds'))
```

Read the peptide data into a QFeatures object

We now start reading in the peptide-level quantification data from MaxQuant into a **QFeatures** object.

Defining the contaminant proteins

We need to remove contaminant proteins. Below, we parse the MaxQuant contaminants fasta to extract the IDs for the proteins.

```
contaminant_proteins <- get_maxquant_cont_accessions()
```

```
## No contaminants fasta supplied, defaulting to contaminants.fasta.gz from biomasslmb
```

Reading in the peptide-level quantification data

We need to read in both the proteinGroups.txt files from MaxQuant to identify the proteins which pass the 1% FDR threshold, and the modificationSpecificPeptides.txt file to get the peptide-level quantification.

```

pg_inf <- here('raw/HALO/proteinGroups.txt.gz')
pg_df <- read.delim(pg_inf)

seq2razor <- read.delim(here('raw/HALO/peptides.txt.gz')) %>%
  select(Sequence, Leading.razor.protein)

pep_inf <- here('raw/HALO/modificationSpecificPeptides.txt.gz')
pep_df <- read.delim(pep_inf)

# Merge with the leading razor protein column
pep_df <- merge(pep_df, seq2razor, by='Sequence')

per2_lines <- which(grepl('015055', pep_df$Proteins))
halo_lines <- which(pep_df$Proteins=='PER2HALO;PER2MNG')

pep_df$Leading.razor.protein[halo_lines] <- 'HALO_TAG'
pep_df$Leading.razor.protein[per2_lines] <- '015055'

# Merge with the protein details and Q-values
pep_df <- merge(pep_df, uniprot2details,
  by.x='Leading.razor.protein', by.y='UniprotID', all.x=TRUE) %>%
  merge(pg_df[,c('Majority.protein.IDs', 'Q.value')], 'Leading.razor.protein',
    by.y='Majority.protein.IDs', all.x=TRUE)

```

Extracting the experiment to sample mapping

We can extract the details for the experiment to sample mapping from the mqpar.xml file

```

mqpar_inf <- here('raw/HALO/mqpar.xml')
mqpar <- xml2::read_xml(mqpar_inf)

# Extract the file paths and experiment names
filepaths <- strsplit(
  xml_text(xml_find_all(mqpar, ".*//filePaths")), split='F:\\\\')[[1]]
filepaths <- filepaths[filepaths!='']

# Extract sample names from file paths
sample <- gsub(
  '(2565608863_Andrei_PER2_121225\\\\\\\\2565608863_Andrei_|.raw)', '', filepaths)

experiments <- strsplit(
  xml_text(xml_find_all(mqpar, ".*//experiments")), split='')[[1]]

# Checking that the lengths match
length(sample)==length(experiments)

## [1] TRUE

```

```

# Create the coldata dataframe
coldata <- data.frame('sample'=sample, 'experiments'=experiments) %>%
  separate(sample, into=c('sample', NA)) %>%

```

```

distinct() %>%
mutate(sample=sub('(HA|P2H)', '\\1_', sample)) %>%
separate(sample, into=c('condition', 'replicate'), sep='_', remove=FALSE) %>%
mutate(quantCols=sample) %>%
tibble::column_to_rownames('sample')

# named list to map from experimental name to sample name
exp2sample <- coldata$quantCols
names(exp2sample) <- coldata$experiments

coldata %>%
  knitr::kable()

```

	condition	replicate	experiments	quantCols
HA_1	HA	1	A	HA_1
HA_2	HA	2	B	HA_2
HA_3	HA	3	C	HA_3
P2H_1	P2H	1	D	P2H_1
P2H_2	P2H	2	F	P2H_2
P2H_3	P2H	3	G	P2H_3

Creating the QFeatures object

Now we can finally create the QFeatures object

```

# Define the columns corresponding to the abundance values
abundance_cols <- 32:37

# Rename the abundance columns to the sample names
colnames(pep_df)[abundance_cols] <- gsub(
  'Intensity.', '', colnames(pep_df)[abundance_cols])

colnames(pep_df)[abundance_cols] <- sapply(
  colnames(pep_df)[abundance_cols], function(x) exp2sample[[x]])

# Reorder the coldata to match the order of the abundance columns
coldata <- coldata[colnames(pep_df)[abundance_cols],]

# Create the QFeatures object
lfq_qf <- readQFeatures(assayData = pep_df,
  colData = coldata,
  quantCols = abundance_cols,
  name = 'peptides_raw')

```

```
## Checking arguments.
```

```
## Loading data as a 'SummarizedExperiment' object.
```

```
## Formatting sample annotations (colData).
```

```
## Formatting data as a 'QFeatures' object.
```

```
# Set the colData for the peptides_raw experiment  
colData(lfq_qf[['peptides_raw']]) <- colData(lfq_qf)
```

MaxQuant has reported 107855 quantification values as zero. These should be replaced with NA, since mass spectrometry is not capable of asserting that the protein had zero abundance in the sample.

```
lfq_qf[['peptides_raw']] <- QFeatures::zeroIsNA(lfq_qf[['peptides_raw']])
```

Filter the peptide-level quantification data

We will now filter the peptide-level quantification data to remove contaminants and proteins which do not pass the 1% FDR threshold.

```
lfq_qf[['peptides_filtered']] <- biomasslmb::filter_features_mq_dda(  
  lfq_qf[['peptides_raw']],  
  contaminant_proteins=contaminant_proteins,  
  filter_contaminant=TRUE,  
  filter_associated_contaminant=TRUE,  
  unique_master=FALSE)
```

```
## Filtering data...
```

```
## 35885 features found from 4088 master proteins => Input
```

```
## 35816 features found from 4046 master proteins => Removed hits to decoy database
```

```
## 246 contaminant proteins supplied
```

```
## 926 proteins identified as 'contaminant associated'
```

```
## 35093 features found from 4003 master proteins => contaminant features removed
```

```
## 34816 features found from 3979 master proteins => associated contaminant features removed
```

```
## 34816 features found from 3979 master proteins => MaxQuant-labelled 'Contaminants' removed
```

```
## 34816 features found from 3979 master proteins => features without a master protein removed
```

```
## 33326 features found from 3961 master proteins => features without quantification removed
```

```
lfq_qf <- filterFeatures(lfq_qf, i='peptides_filtered', ~ Q.value <= 0.01)
```

```
## 'Q.value' found in 2 out of 2 assay(s).
```

```
biomasslmb:::message_parse(rowData(lfq_qf[['peptides_filtered']]),
                           "Leading.razor.protein",
                           "Proteins with Q-Value > 0.01 removed")
```

```
## 33324 features found from 3959 master proteins => Proteins with Q-Value > 0.01 removed
```

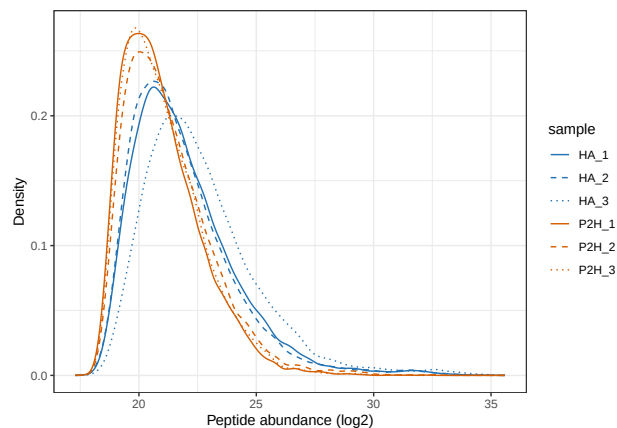
Normalise the peptide-level quantification data

For the normalisation and summarisation, we need our quantification values to be on a log-scale, so we log-transform below

```
lfq_qf[['peptides_filtered']] <- QFeatures::logTransform(
  lfq_qf[['peptides_filtered']], base=2)
```

Next, we plot the peptide intensities.

```
# Plot the peptide-level quantification distributions per sample
plot_quant(lfq_qf[['peptides_filtered']], log2transform=FALSE, method='density') +
  scale_colour_manual(values=rep(get_cat_palette(4), each=3)) +
  aes(linetype=sample) +
  scale_linetype_manual(values=rep(1:3, times=4)) +
  xlab('Peptide abundance (log2)')
```



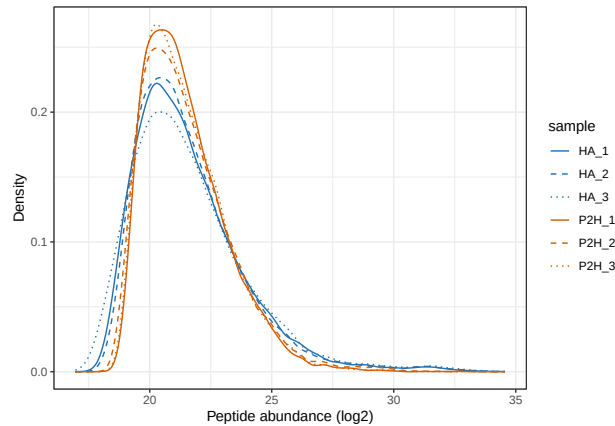
Normalise

Since the same amount of sample was injected for each sample, we could normalise to remove differences in the overall distribution of abundances. This is performed below using median normalisation.

```
lfq_qf[['peptides_norm']] <- QFeatures::normalize(lfq_qf[['peptides_filtered']],
                                                  method='diff.median')
```

Below, we can check this normalisation step has worked as intended.

```
# Plot the peptide-level quantification distributions per sample
plot_quant(lfq_qf[['peptides_norm']], log2transform=FALSE, method='density') +
  scale_colour_manual(values=rep(get_cat_palette(4), each=3)) +
  aes(linetype=sample) +
  scale_linetype_manual(values=rep(1:3, times=4)) +
  xlab('Peptide abundance (log2)')
```



Normalise to histone proteins

Median normalisation is suitable when we expect the overall distribution of protein abundances to be similar between the conditions. However, we don't expect the overall distribution of protein abundances to necessarily be similar between the conditions here, since one is a free HALO tag and the other is PER2-HALO. We should also consider normalising to a set of proteins which we expect to be constant across the samples. Histone proteins are often used for this purpose since they are generally stable in abundance. Below, we identify the histone proteins in our dataset and explore their abundance post normalisation. They are more abundant in the HALO condition, suggesting that global median normalisation may be more inappropriate.

```
hist_df <- uniprot2details %>%
  filter(grepl('^Histone H', Protein.names))

hist_df %>%
  select('UniprotID'=UniprotID, 'Gene Name'=Gene.Names.First, 'Protein names'=Protein.names) %>%
  knitr::kable()
```

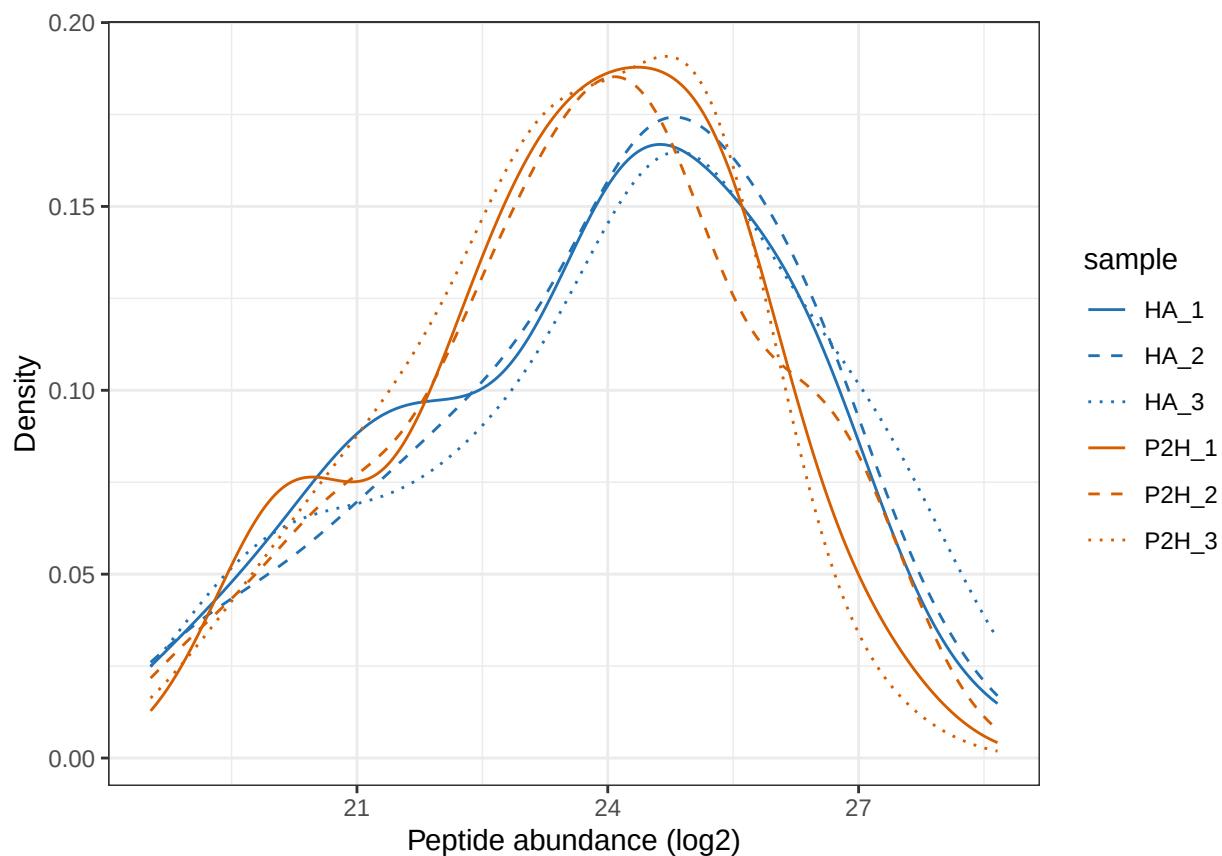
Gene	
UniprotName	Protein names
P07305H1-0	Histone H1.0 (Histone H1') (Histone H1(0)) [Cleaved into: Histone H1.0, N-terminally processed]
P10412H1-4	Histone H1.4 (Histone H1b) (Histone H1s-4)
P16104H2AX	Histone H2AX (H2a/x) (Histone H2A.X)
P16403H1-2	Histone H1.2 (Histone H1c) (Histone H1d) (Histone H1s-1)
P49450CENPA	Histone H3-like centromeric protein A (Centromere autoantigen A) (Centromere protein A) (CENP-A)
P58876H2BC5	Histone H2B type 1-D (H2B-clustered histone 5) (HIRA-interacting protein 2) (Histone H2B.1 B) (Histone H2B.b) (H2B/b)
P62805H4C1	Histone H4

Gene	Protein names
UniprotName	
P62807H2BC4	Histone H2B type 1-C/E/F/G/I (Histone H2B.1 A) (Histone H2B.a) (H2B/a) (Histone H2B.g) (H2B/g) (Histone H2B.h) (H2B/h) (Histone H2B.k) (H2B/k) (Histone H2B.l) (H2B/l)
Q02539H1-1	Histone H1.1 (Histone H1a)
Q16778H2BC21	Histone H2B type 2-E (H2B-clustered histone 21) (Histone H2B-GL105) (Histone H2B.q) (H2B/q)
Q5TEC6H3-7	Histone H3-7
Q71DI3H3C15	Histone H3.2 (H3-clustered histone 13) (H3-clustered histone 14) (H3-clustered histone 15) (Histone H3/m) (Histone H3/o)
Q71UI9H2AZ2	Histone H2A.V (H2A.F/Z) (H2A.Z variant histone 2)
Q8IUE6H2AC21	Histone H2A type 2-B (H2A-clustered histone 21)
Q8N257H2BC26	Histone H2B type 3-B (H2B type 12) (H2B-clustered histone 26) (H2B.U histone 1)
Q92522H1-10	Histone H1.10 (Histone H1x)
Q93077H2AC6	Histone H2A type 1-C (H2A-clustered histone 6) (Histone H2A/l)
Q96A08H2BC1	Histone H2B type 1-A (Histone H2B, testis) (TSH2B.1) (hTSH2B) (Testis-specific histone H2B)
Q99878H2AC14	Histone H2A type 1-J (Histone H2A/e)
Q99879H2BC14	Histone H2B type 1-M (Histone H2B.e) (H2B/e)
Q99880H2BC13	Histone H2B type 1-L (Histone H2B.c) (H2B/c)

```
hist <- hist_df %>%
  pull(UniprotID)

# Plot the peptide-level quantification distributions per sample
plot_quant(lfq_qf[['peptides_norm']][
  rowData(lfq_qf[['peptides_norm']])$Leading.razor.protein %in% hist,],
  log2transform=FALSE, method='density') +
  scale_colour_manual(values=rep(get_cat_palette(4), each=3)) +
  aes(linetype=sample) +
  scale_linetype_manual(values=rep(1:3, times=4)) +
  xlab('Peptide abundance (log2)')

## Warning: Removed 86 rows containing non-finite outside the scale range
## ('stat_density()').
```



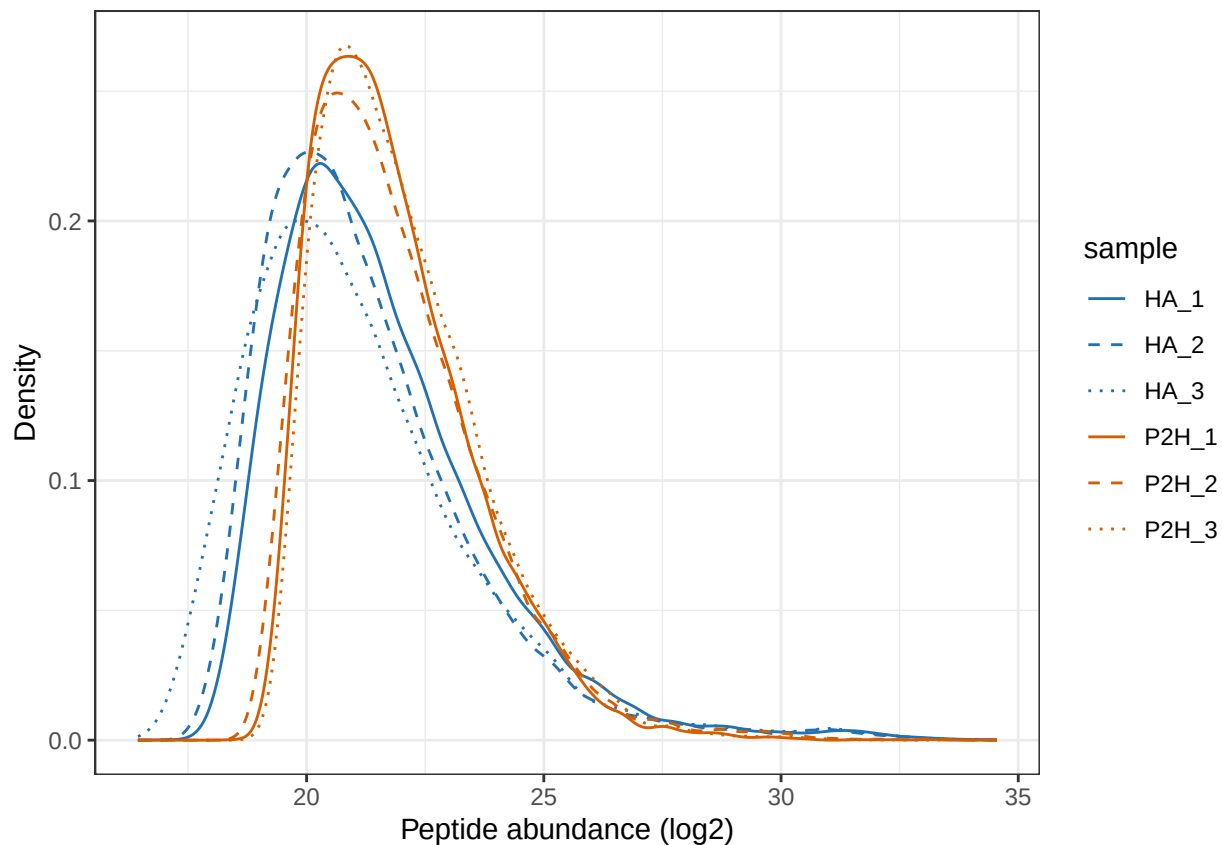
Below, we normalise to the histone proteins and replot the peptide-level abundance distributions.

```
hist_medians <- get_medians(lfq_qf[['peptides_filtered']][
  rowData(lfq_qf[['peptides_filtered']])$Leading.razor.protein %in% hist,])

lfq_qf[['peptides_norm_hist']] <- center_normalise_to_ref(
  lfq_qf[['peptides_filtered']], medians = hist_medians, on_log_scale = TRUE)

# Plot the peptide-level quantification distributions per sample
plot_quant(lfq_qf[['peptides_norm_hist']], log2transform=FALSE, method='density') +
  scale_colour_manual(values=rep(get_cat_palette(4), each=3)) +
  aes(linetype=sample) +
  scale_linetype_manual(values=rep(1:3, times=4)) +
  xlab('Peptide abundance (log2)')
```

```
## Warning: Removed 96622 rows containing non-finite outside the scale range
## ('stat_density()').
```



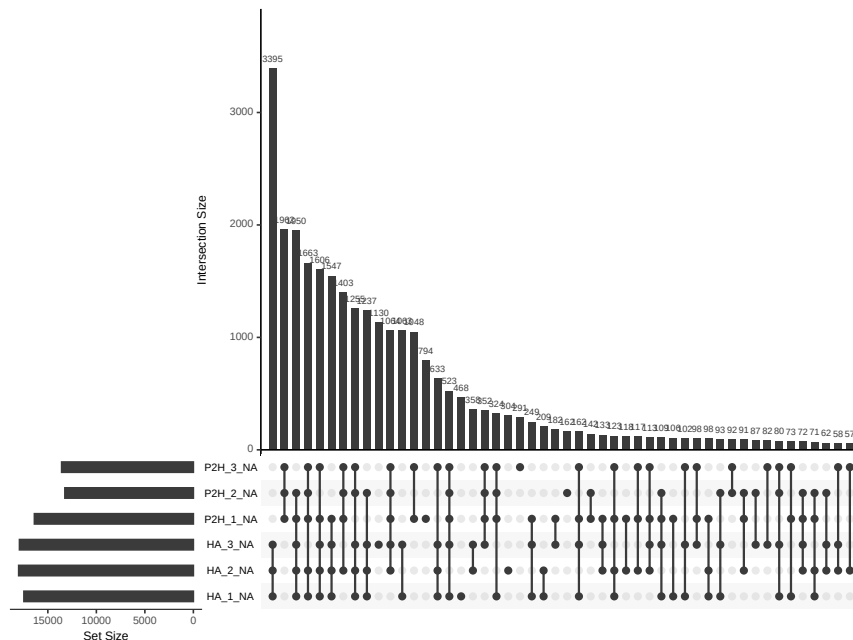
Removing peptides with too many missing values

For the estimation of protein-level abundances in the next notebook, we will use `limpa`. We do not need to exclude peptides with any missing values for this. However, we still don't want to retain peptides with too many missing values, since these will not be very informative in estimating the protein-level quantification.

Below, we inspect the most common patterns for the missing values.

The missingness is sometimes consigned to all samples from a condition(s). It looks like the missingness is a mixture of missing at random (MAR) and missing at random (MNAR) and is likely partly due to falling below detection limits.

```
plot_missing_upset(lfq_qf, i='peptides_norm_hist')
```



Here, we'll set a limit of maximum 4/6 missing values per peptide.

```
table(nNA(lfq_qf[['peptides_norm_hist']])$nNArows$nNA)
```

```
##
##      0      1      2      3      4      5
## 5272 3149 3683 6834 6325 8061
```

```
lfq_qf[['peptides_filtered_missing']] <- QFeatures::filterNA(
  lfq_qf[['peptides_norm_hist']], 4/6)

biomasslmb:::message_parse(rowData(lfq_qf[['peptides_filtered_missing']]),
  'Leading.razor.protein',
  "Removing peptides with more than 4/6 missing values")
```

```
## 25263 features found from 3684 master proteins => Removing peptides with more than 4/6 missing values
```

Inspecting the number of peptides and proteins through the processing steps

Now that we have performed all the filtering steps, it's helpful to visualise how many peptides/proteins were retained at each level of the processing.

Samples per peptide

We'll start by inspecting the number of peptides in each experiment. We set the row variables to be **Sequence** and **Modifications** so that we count the number of unique peptides.

```

rename_cols <- c('All peptides' = 'peptides_raw' ,
                'Quantified, contaminants removed' = 'peptides_filtered',
                'Max 4/6 missing values' = 'peptides_filtered_missing')

rowvars <- c('Sequence', 'Modifications')
rowDataNames(lfq_qf)[['peptides_raw']]

```

```

## [1] "Leading.razor.protein"      "Sequence"
## [3] "Modifications"             "Mass"
## [5] "Mass.Fractional.Part"     "Protein.Groups"
## [7] "Proteins"                  "Gene.Names.x"
## [9] "Protein.Names"             "Unique..Groups."
## [11] "Unique..Proteins."         "Acetyl..Protein.N.term."
## [13] "Oxidation..M."             "Missed.cleavages"
## [15] "Experiment.A"              "Experiment.B"
## [17] "Experiment.C"              "Experiment.D"
## [19] "Experiment.F"              "Experiment.G"
## [21] "Retention.time"            "Calibrated.retention.time"
## [23] "Charges"                   "PEP"
## [25] "MS.MS.scan.number"         "Raw.file"
## [27] "Score"                     "Delta.score"
## [29] "Reverse"                   "Potential.contaminant"
## [31] "Intensity"                 "id"
## [33] "Protein.group.IDs"         "Peptide.ID"
## [35] "Evidence.IDs"              "MS.MS.IDs"
## [37] "Best.MS.MS"                "Oxidation..M..site.IDs"
## [39] "MS.MS.Count"               "Taxonomy.IDs"
## [41] "Taxonomy.names"            "Mass.deficit"
## [43] "Entry.Name"                "Reviewed"
## [45] "Protein.names"             "Gene.Names.y"
## [47] "Organism"                  "Length"
## [49] "Gene.Names.First"          "Q.value"

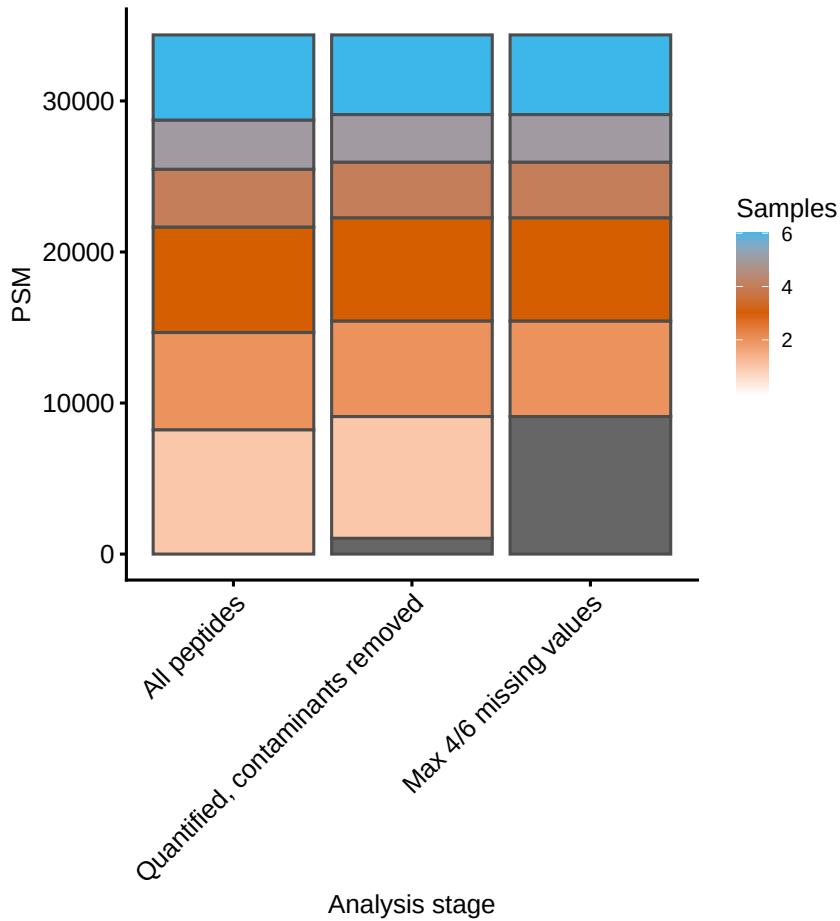
```

```

samples_present <- get_samples_present(
  lfq_qf[, , unname(rename_cols)], rowvars, rename_cols)

plot_samples_present(samples_present, rowvars, breaks=seq(2,10,2)) + ylab('PSM')

```

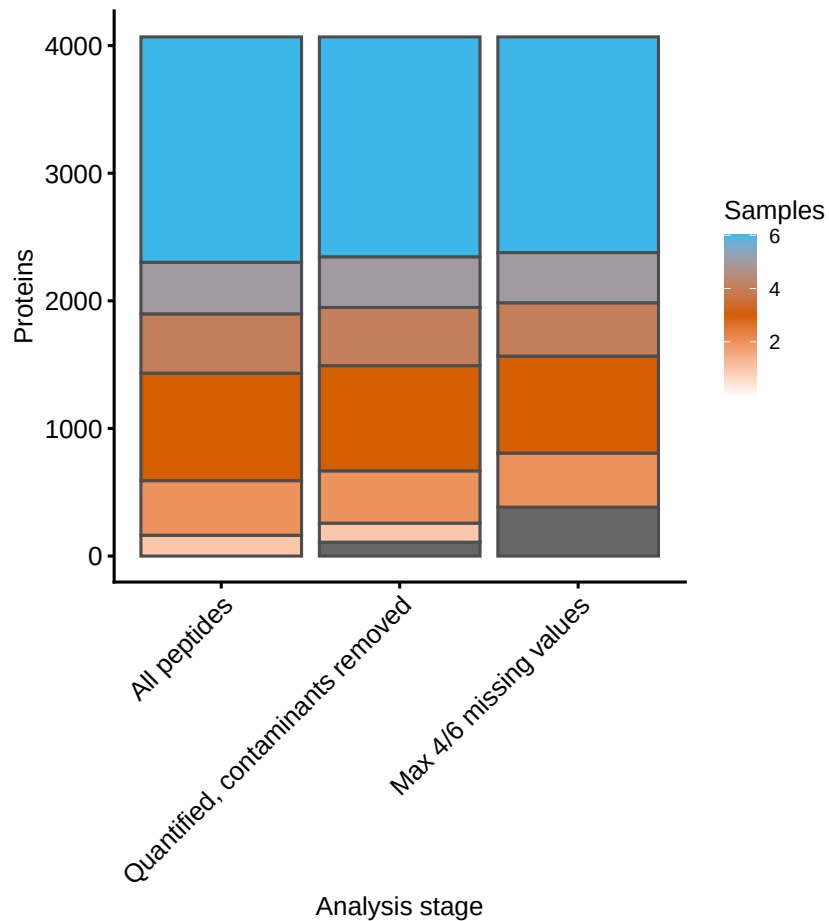


Samples per Protein

Next, we'll use the same functions to inspect the number of proteins at each level of processing. We need to set the row variables to be just the `Leading.razor.protein` column.

```
rowvars_prot <- c('Leading.razor.protein')

samples_present <- get_samples_present(lfq_qf, rowvars_prot, rename_cols)
plot_samples_present(samples_present, rowvars_prot, breaks=seq(2,10,2))
```



```
output_dir <- here('notebooks/output')
```

```
saveRDS(lfq_qf, here(output_dir, 'HALO_lfq_qf.rds'))
```

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Sonoma 14.7
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] stats4 stats graphics grDevices utils datasets methods
## [8] base
```

```

##
## other attached packages:
## [1] xml2_1.4.0                QFeatures_1.18.0
## [3] MultiAssayExperiment_1.34.0 SummarizedExperiment_1.38.1
## [5] Biobase_2.68.0            GenomicRanges_1.60.0
## [7] GenomeInfoDb_1.44.2       IRanges_2.42.0
## [9] S4Vectors_0.48.0         BiocGenerics_0.54.0
## [11] generics_0.1.4           MatrixGenerics_1.20.0
## [13] matrixStats_1.5.0        here_1.0.2
## [15] dplyr_1.1.4              tidyr_1.3.1
## [17] ggplot2_4.0.0            biomasslmb_0.0.4
##
## loaded via a namespace (and not attached):
## [1] DBI_1.2.3                 gridExtra_2.3          rlang_1.1.6
## [4] magrittr_2.0.4           clue_0.3-66            compiler_4.5.2
## [7] RSQLite_2.4.3            png_0.1-8              vctrs_0.6.5
## [10] reshape2_1.4.4           stringr_1.5.2          ProtGenerics_1.40.0
## [13] pkgconfig_2.0.3          crayon_1.5.3           fastmap_1.2.0
## [16] backports_1.5.0          XVector_0.48.0         labeling_0.4.3
## [19] rmarkdown_2.29           UCSC.utils_1.4.0       visdat_0.6.0
## [22] UpSetR_1.4.0             purrr_1.1.0            bit_4.6.0
## [25] xfun_0.54                cachem_1.1.0           jsonlite_2.0.0
## [28] blob_1.2.4              DelayedArray_0.34.1    cluster_2.1.8.1
## [31] R6_2.6.1                 stringi_1.8.7          RColorBrewer_1.1-3
## [34] genefilter_1.90.0        Rcpp_1.1.0             knitr_1.50
## [37] BiocBaseUtils_1.10.0     Matrix_1.7-4           splines_4.5.2
## [40] igraph_2.1.4            tidyselect_1.2.1       rstudioapi_0.17.1
## [43] abind_1.4-8             yaml_2.3.10            lattice_0.22-7
## [46] tibble_3.3.0            plyr_1.8.9             withr_3.0.2
## [49] KEGGREST_1.48.1         S7_0.2.0              evaluate_1.0.5
## [52] uniprotREST_1.0.0        survival_3.8-3         Biostrings_2.76.0
## [55] pillar_1.11.1           corrrplot_0.95         checkmate_2.3.3
## [58] rprojroot_2.1.1         scales_1.4.0           xtable_1.8-4
## [61] glue_1.8.0              lazyeval_0.2.2         tools_4.5.2
## [64] robustbase_0.99-6       annotate_1.86.1         XML_3.99-0.19
## [67] grid_4.5.2              MsCoreUtils_1.20.0     AnnotationDbi_1.70.0
## [70] GenomeInfoDbData_1.2.14 nanianr_1.1.0          cli_3.6.5
## [73] S4Arrays_1.8.1          AnnotationFilter_1.32.0 gtable_0.3.6
## [76] DEoptimR_1.1-4          digest_0.6.37          SparseArray_1.8.1
## [79] farver_2.1.2            memoise_2.0.1          htmltools_0.5.8.1
## [82] lifecycle_1.0.4         httr_1.4.7            bit64_4.6.0-1
## [85] MASS_7.3-65

```